National University
of Computer & Emerging Sciences,
Peshawar Campus

| | | |
|---|---|---|
| **Program:** BS (AI 5A) | **Exam:** Assignment #01 | **Total Marks:** |
| **Semester:** Fall-2026 | **Course:** CN | **Date:** |
| **Name:** | **Reg. No:** | |

# Assignment No. 1

## University Research Opportunity Portal

## 1 Scenario

Your university currently shares research opportunities through emails, WhatsApp groups, and noticeboards. Because the information is scattered across different platforms, many students miss valuable opportunities, while faculty members find it difficult to manage their research openings.

The university has therefore decided to develop a Research Opportunity Portal where faculty members can post, view, update, and manage research opportunities in one place.

You are working as a junior software developer in the university's software development team. Your responsibility is to develop a complete web-based application consisting of a backend REST API, database, and a simple frontend interface for managing research opportunities.

## 2 Backend Requirements

Develop a backend REST API that allows faculty members to perform CRUD operations on research opportunities.

Each research opportunity should contain the following information:

- Unique ID
- Research title
- Research description
- Research area

- Faculty member's name
- Department
- Required skills
- Number of available positions
- Application deadline
- Status: Open or Closed

Students must implement the following operations.

### 2.1 1. Create a Research Opportunity

Create and store a new research opportunity in the database.

POST /api/opportunities

### 2.2 2. Retrieve All Research Opportunities

Return all research opportunities stored in the database.

GET /api/opportunities

### 2.3 3. Retrieve One Research Opportunity

Return the complete details of a specific opportunity using its ID.

GET /api/opportunities/:id

### 2.4 4. Update a Research Opportunity

Update an existing opportunity, such as its title, required skills, available positions, application deadline, or status.

PUT /api/opportunities/:id

### 2.5 5. Delete a Research Opportunity

Delete a research opportunity that was posted by mistake or is no longer required.

DELETE /api/opportunities/:id

### 2.6 Status Codes

Students should use appropriate HTTP status codes, including:

- `200 OK`
- `201 Created`
- `400 Bad Request`
- `404 Not Found`
- `500 Internal Server Error`

## 3 Frontend Requirements

Students must develop a simple frontend interface for interacting with the backend API.

The frontend should provide the following functionality:

1. Display all available research opportunities.
2. Display complete details of a selected research opportunity.
3. Provide a form for creating a new research opportunity.
4. Provide an option to update an existing research opportunity.
5. Allow the status of an opportunity to be changed from `Open` to `Closed`.
6. Provide an option to delete a research opportunity.
7. Display appropriate success and error messages.
8. Perform basic validation for required form fields.

The frontend must communicate with the backend through the implemented REST API. Data should not be hard-coded in the frontend.

Students are not required to create an advanced or highly complex user interface. A simple, clean, and functional interface is sufficient.

## 4 Technologies

Students may use suitable technologies for developing the application.

### 4.1 Backend

Students may use any one of the following:

- Python
- Java
- JavaScript / Node.js

### 4.2 Database

Students may use:

- MySQL

### 4.3 Frontend

Students may use any suitable frontend technology, including:

- HTML
- CSS
- JavaScript
- Bootstrap
- React.js

## 5 GitHub Repository

Students must create a GitHub repository for the assignment.

The repository must contain the complete project, including:

- Backend source code
- Frontend source code
- Database schema or setup files
- `README.md`
- Exported Postman or Bruno collection

Students should make meaningful Git commits while developing the project instead of uploading the complete project in a single commit at the end.

The GitHub repository should be accessible to the instructor at the time of submission.

The repository link must be included in the README.md file and submitted along with the assignment.

Example:

GitHub Repository:
https://github.com/username/research-opportunity-portal

Do not upload database passwords, API keys, environment variables, personal access tokens, or other private credentials to GitHub.

## 6 Postman or Bruno Testing

Students must test the completed backend API using either **Postman** or **Bruno**.

The testing demonstration must include:

1. Creating at least three research opportunities.
2. Retrieving all research opportunities.
3. Retrieving one opportunity using its ID.
4. Updating an existing opportunity.
5. Changing an opportunity's status from `Open` to `Closed`.
6. Deleting an opportunity.
7. Requesting the deleted opportunity again to demonstrate a `404 Not Found` response.
8. Sending one request containing invalid or missing data to demonstrate input validation.

Students must export their Postman or Bruno collection and include it in the final submission and GitHub repository.

## 7 One-Minute Demonstration Video

Students must record a video of approximately one minute demonstrating:

- The backend server running.
- The frontend application running.
- Creating a research opportunity through the application.
- Retrieving and displaying stored opportunities.
- Updating an opportunity.
- Closing an opportunity.
- Deleting an opportunity.
- Displaying at least one validation or `404 Not Found` error.
- API requests and responses in Postman or Bruno.

## 8 Submission Requirements

Submit one ZIP file containing:

- Complete frontend and backend source code
- Database setup/schema
- Exported Postman or Bruno collection
- `README.md` with setup and run instructions
- GitHub repository link
- One-minute demonstration video or video link

The ZIP file must be named as:

`P24_0000_NAME_CLASS.zip`

## 9 Important Instructions

- This is an individual assignment.
- Both frontend and backend are required.
- All CRUD operations must use the database.
- Do not use hard-coded data.
- The project must run using the instructions in `README.md`.
- Keep the GitHub repository accessible to the instructor.
- Do not upload passwords, API keys, or other credentials.
- The demonstration video must not exceed one minute.
- Copied or plagiarized work may receive zero marks.

---

**End of Assignment**